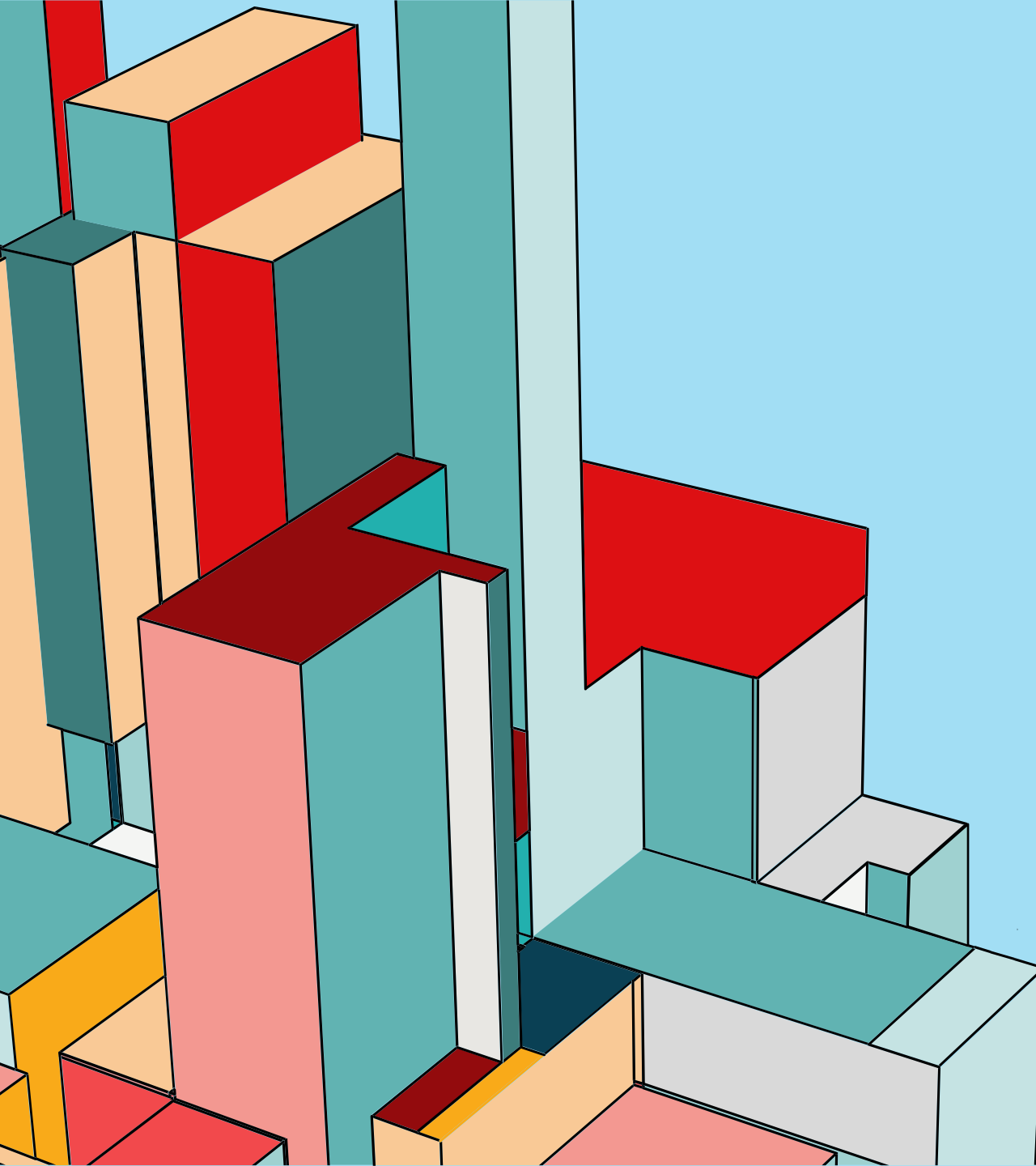




LECTURE 19 **DYNAMIC PROGRAMMING IV**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

TODAY'S AGENDA

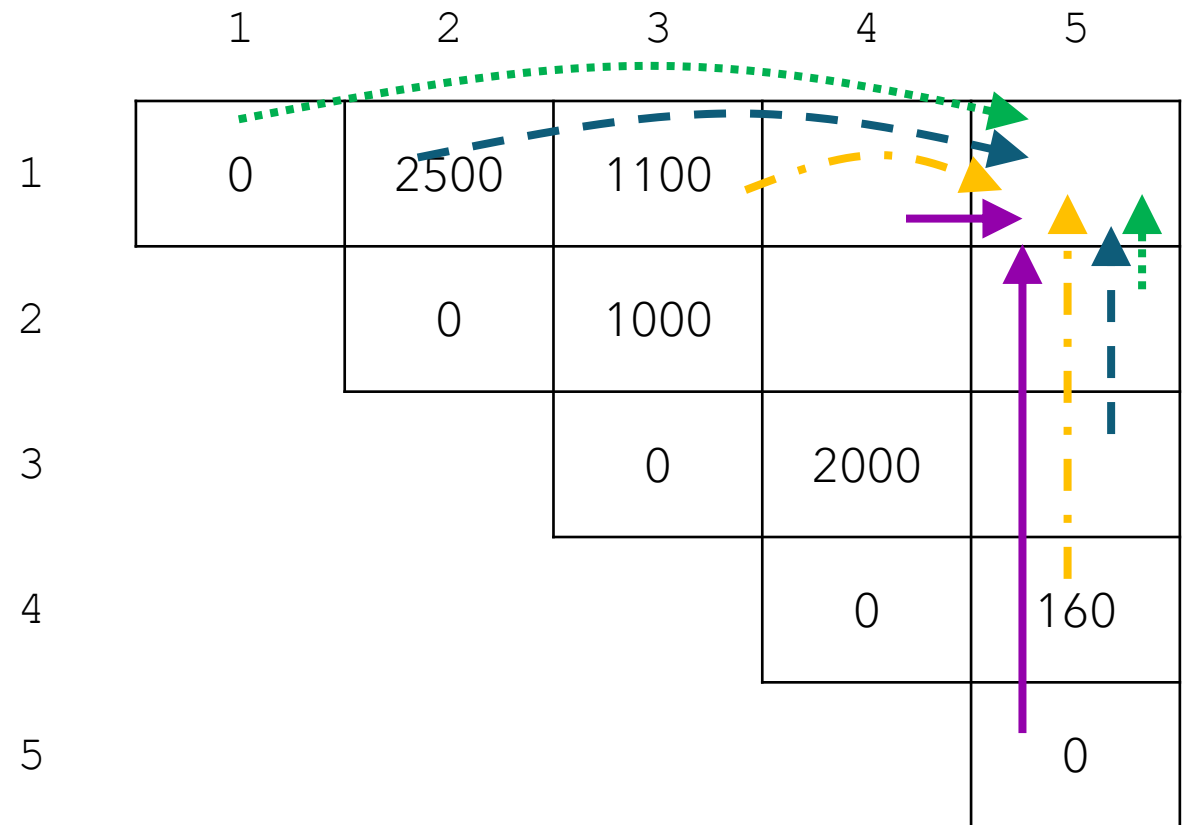
- Memoization
- The Floyd-Warshall Algorithm

SKIPPING SOME SUBPROBLEMS

Last week, someone asked an important question: Do we need to calculate answers to all of the subproblems?

For many dynamic programming problems, the answer is yes. Otherwise we risk missing an optimal solution.

In matrix chaining, the final box of the DP table depends, at least indirectly, upon every other value in the table.



RETURN TO KNAPSACK

	0	1	2	...	W-2	W-1	W
0	0	0	0	0	0	0	0
1	0						
2	0						
...	0						
n-2	0						
n-1	0						
n	0						Target!

Last week, we discussed the knapsack problem – it was our first example of a problem needing a 2-D DP array.

Our knapsack implementation filled in every entry in its table. Was that really necessary?

Weights	2	3	4	5
Values	2	4	5	7

SKIPPING SOME SUBPROBLEMS

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	2	2	2	2	2	2	2	2	2
2	0	0	2	4	4	6	6	6	6	6	6
3	0	0	2	4	5	6	7	9	9	11	11
4	0	0	2	4	5	7	7	9	11	12	13

$$OPT(n, W) = \begin{cases} OPT(n-1, W) & \text{if } w_n > W \\ \max(OPT(n-1, W), v_n + OPT(n-1, W - w_n)) & \text{otherwise} \end{cases}$$

Note that, given this recurrence, we will NEVER need $OPT(3, 9)$.

Nor anything in row 4 other than the final value, nor...

Weights	2	3	4	5
Values	2	4	5	7

SUBMITTY POLL #2

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	2	2	2	2	2	2	2	2	2
2	0	0	2	4	4	6	6	6	6	6	6
3	0	0	2	4	5	6	7	9	9	11	11
4	0	0	2	4	5	7	7	9	11	12	13

How many table entries do we actually need to fill in? (Don't count base cases.)

Answer in the Submitty poll now.



MEMOIZATION

The structure of the knapsack problem is such that, with certain inputs, the answers to particular subproblems never come up.

The standard “bottom-up” approach to dynamic programming doesn’t have any sort of mechanism for bypassing these problems, though – in fact, we can’t know which smaller problems are skippable until we examine the larger problems.

MEMOIZATION

Note that a “top-down” recursive approach doesn’t have this problem – it only makes recursive calls for the problems it actually needs to solve.

The problem with recursion is that it will make the same recursive call multiple times.

We can combine both approaches using *memoization* – record each recursive answer in a hashtable, and check if the answer exists before each potential recursive call.

MEMOIZATION

$$\begin{aligned} &OPT(n, W) \\ &= \begin{cases} OPT(n-1, W) & \text{if } w_n > W \\ \max(OPT(n-1, W), v_n + OPT(n-1, W - w_n)) & \text{otherwise} \end{cases} \end{aligned}$$

The standard top-down recursive code looks like this:

```
def knap(n,W,vals,wts):  
    if n==0 or W==0: return 0  
    a = knap(n-1,W,vals,wts)  
    if wts[n] > W: return a  
    b = knap(n-1,W-wts[n],vals,wts)  
    return max(a,b+vals[n])
```

MEMOIZATION

The *memoized* recursive code looks like this:

```
def knap(n,W,vals,wt):
    if n==0 or W==0: return 0
    a = H.get((n-1,W))      # H is a hashtable; we use the
    if a == NULL:           # tuple (n-1,W) as a key
        a = knap(n-1,W,vals,wt)
        H.store((n-1,W),a)
    if wt[n] > W: return a
    b = H.get((n-1,W-wt[n]))
    if b == NULL:
        b = knap(n-1,W-wt[n],vals,wt)
        H.store((n-1,W-wt[n]),b)
    return max(a,b+vals[n])
```

Weights	2	3	4	5
Values	2	4	5	7

MEMOIZED KNAPSACK

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0										
2	0										
3	0										
4	0										

Let's trace the recursion.

Weights	2	3	4	5
Values	2	4	5	7

MEMOIZED KNAPSACK

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0										2
2	0										
3	0										
4	0										

Let's trace the recursion.

Weights	2	3	4	5
Values	2	4	5	7

MEMOIZED KNAPSACK

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0							2			2
2	0										6
3	0										
4	0										

Let's trace the recursion.

Weights	2	3	4	5
Values	2	4	5	7

MEMOIZED KNAPSACK

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0			2			2	2			2
2	0						6				6
3	0										11
4	0										

Let's trace the recursion.

Weights	2	3	4	5
Values	2	4	5	7

MEMOIZED KNAPSACK

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	2	2		2	2	2			2
2	0	0				6	6				6
3	0					6					11
4	0										13

Let's trace the recursion.

In total, we only calculate 14 of the 40 possible subproblems!

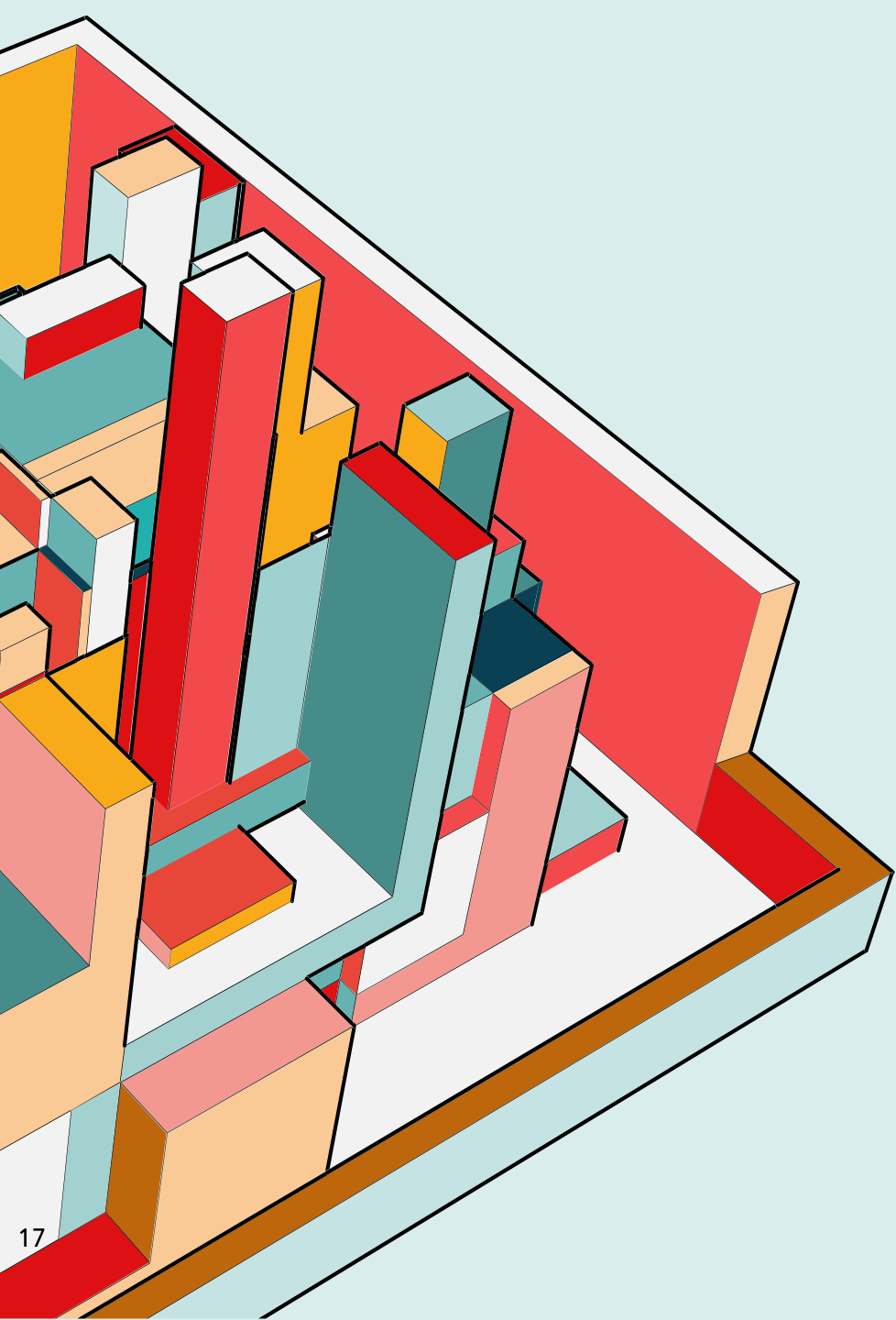
SUMMARY OF MEMOIZATION

A technique for allowing us to address the subproblems of a dynamic programming algorithm in a top-down fashion

Each recursive call is made one time – if we need that value again after that, we have *memoized* it (recorded it in a hashtable or similar data structure).

Only the subproblems whose values are needed are considered – the rest are skipped.

The cost is the overhead of maintaining a hashtable instead of the standard array – well worth it if we can skip a significant number of subproblems.



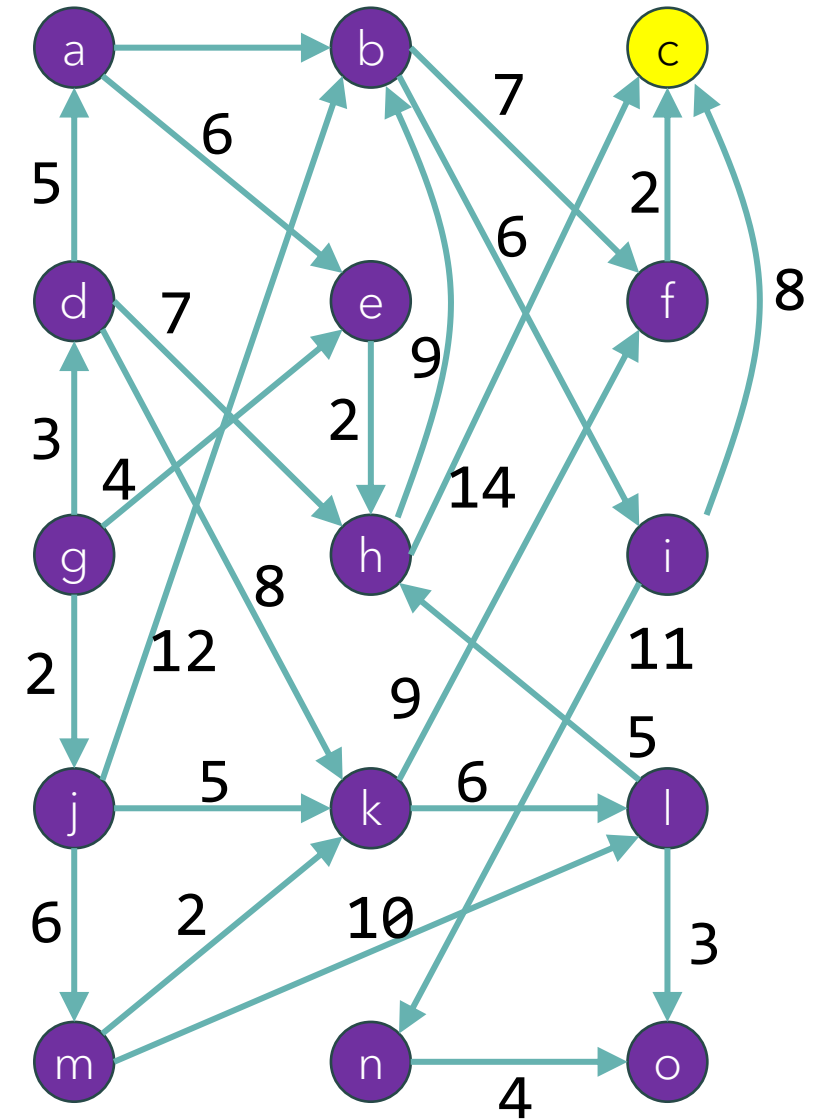
TRANSITIVE CLOSURE AND ALL- PAIRS SHORTEST PATHS

CAN WE GET THERE FROM HERE?

In an undirected graph, connectivity is a simple thing – if the graph is connected, all nodes are reachable from all other nodes.

In a directed graph, though, the question of whether v is reachable from u is trickier (and asymmetric).

For example, in the graph at right, o is reachable from a but not vice versa. And a and j cannot reach each other at all.

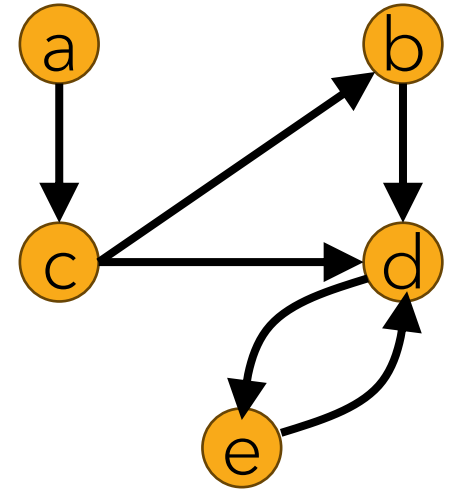


TRANSITIVE CLOSURE

A natural problem is thus to list all of the nodes reachable from each node – this is called the *transitive closure* of the graph.

a:bcde b:de c:bde d:e e:d

Note that this looks awfully like the adjacency list representation of a graph. In fact...



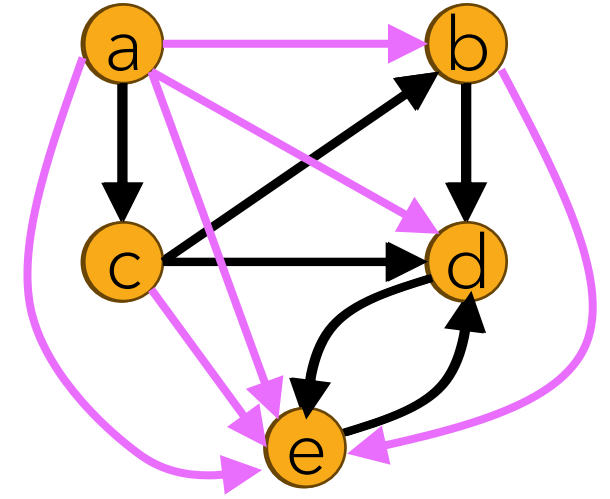
TRANSITIVE CLOSURE

A natural problem is thus to list all of the nodes reachable from each node – this is called the *transitive closure* of the graph.

a:bcde b:de c:bde d:e e:d

Note that this looks awfully like the adjacency list representation of a graph. In fact...

... we can just take the original graph and add a direct edge between node u and node v if v is reachable from u .



What algorithm should we use to generate the transitive closure of a directed graph?

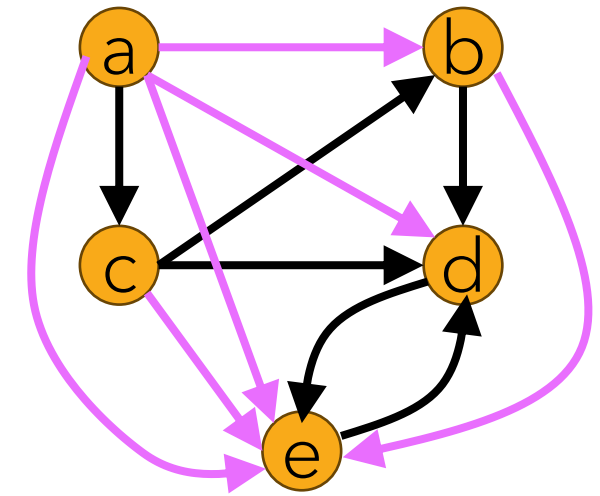
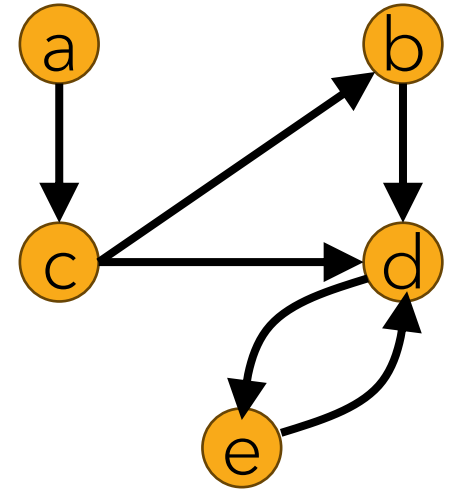
TRANSITIVE CLOSURE

The best way to find the nodes reachable from any single node is just to run BFS or DFS.

So, one obvious approach for the transitive closure would be to run BFS or DFS from each node in the graph, with runtime $O(V^2 + VE)$.

Hopefully this strikes you as inefficient!

The naïve approach ends up traversing most edges many times. There should be a way of reducing this redundancy!

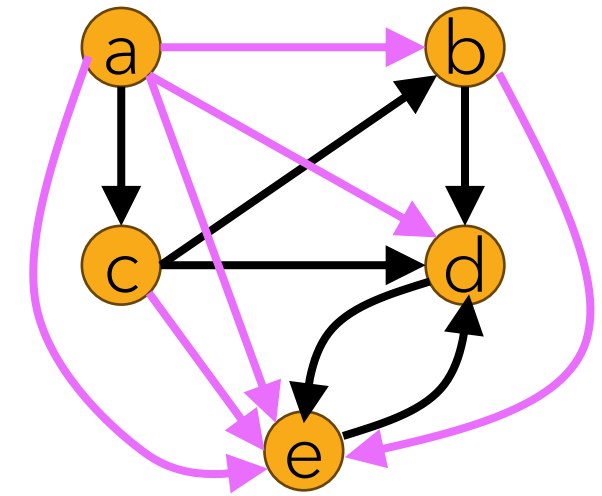
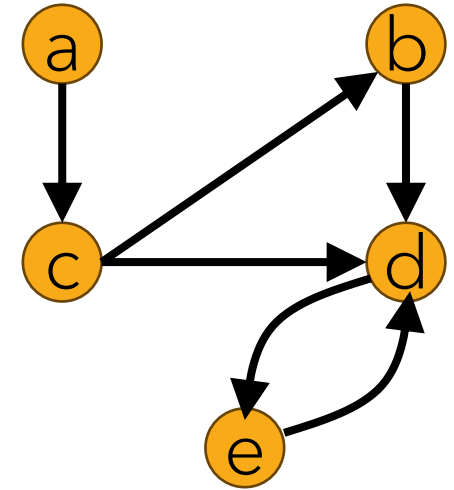


TRANSITIVE CLOSURE

The standard (better) algorithm for finding transitive closure reasons in this fashion: If there is not a direct edge from u to v , then there are some intermediate vertices.

It is tempting to try to find optimal substructure by working with path lengths: first considering paths of length 1, then length 2, ...

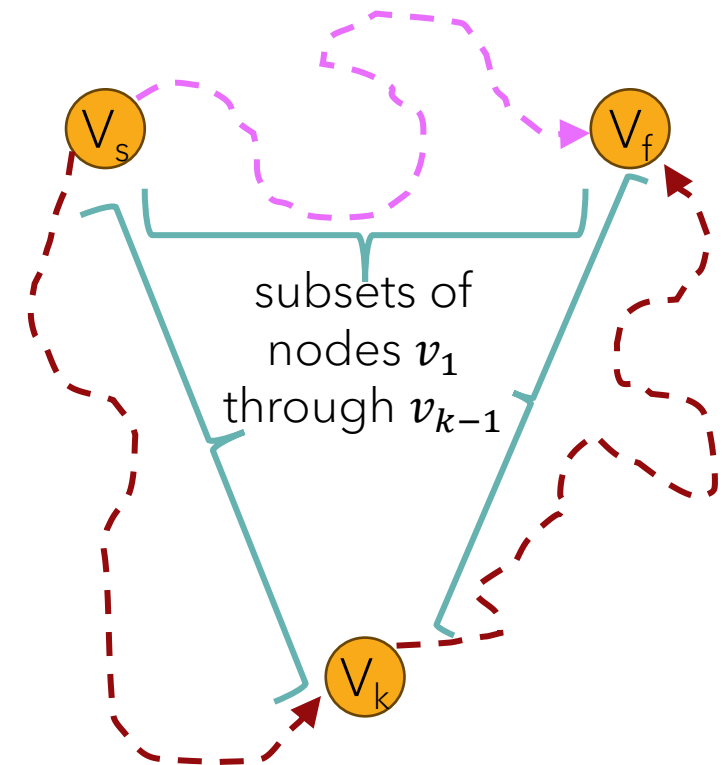
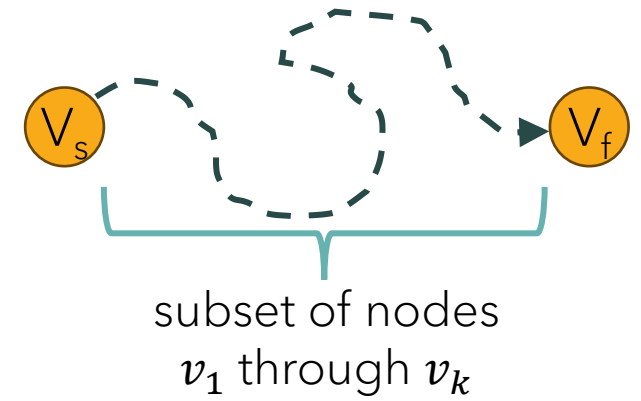
But it turns out to be better to grow the set of allowable intermediate vertices by one node at a time: First node a , then nodes a and/or b , then any nodes from $\{a, b, c\}$, etc.



RECURSIVE SUBSTRUCTURE OF TRANSITIVE CLOSURE

We consider the vertices in some order $v_1, \dots, v_n$. Now, we ask: Is there is a path from v_s to v_f using only intermediate vertices numbered from v_1 to v_k ?

The answer to this question turns out to be yes if and only if: **(a)** There is a path from v_s to v_f using only intermediates v_1 to v_{k-1} , or **(b)** There is a path from v_s to v_k using intermediates v_1 to v_{k-1} AND there is a path from v_k to v_f using these same intermediates.



TRANSITIVE CLOSURE VIA DYNAMIC PROGRAMMING

The previous slide suggests the following recurrence relation (we'll use R for reachable here):

$$R^{(k)}[s, f] = R^{(k-1)}[s, f] \text{ or } (R^{(k-1)}[s, k] \text{ and } R^{(k-1)}[k, f])$$

Notice that our function here is Boolean.

What's the base case for this recurrence relation?

$R^{(0)}[s, f]$ = True if and only if there is a direct edge from v_s to v_f .

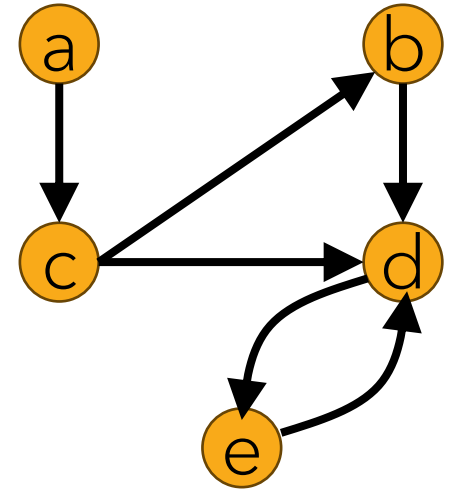
It is useful to observe that our base cases $R^{(0)}[i, j]$ collectively are just the adjacency matrix A of the original graph.

Further – for each k , $R^{(k)}[i, j]$ is the result of adding edges for new connections that specifically need v_k as an intermediate.

EXECUTING THE DYNAMIC PROGRAM

We start with the adjacency matrix:

0	0	1	0	0
0	0	0	1	0
0	1	0	1	0
0	0	0	0	1
0	0	0	1	0



For each round k , we look at one particular vertex and see if we can make new paths using just that vertex.

Mechanically, this involves looking at row k and column k . Where they both have 1s, we will add new 1s to the matrix.

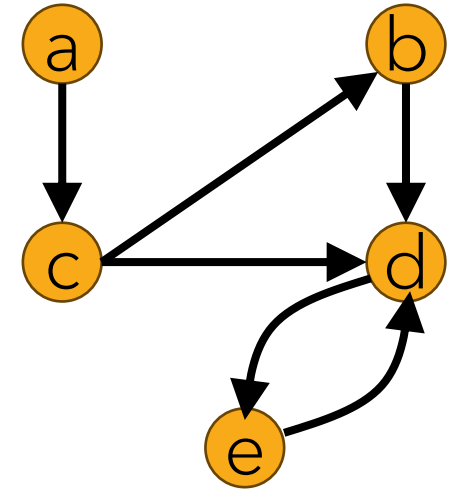
EXECUTING THE DYNAMIC PROGRAM

$k = 1$ (vertex a) adds nothing. For $k = 2$ (vertex b):

0	0	1	0	0
0	0	0	1	0
0	1	0	1	0
0	0	0	0	1
0	0	0	1	0

Here we have a set of matching 1s.

However, the matrix already has a 1 here, so nothing changes. (This reflects the path from c to d via vertex b – since we already had a direct c, d edge, this doesn't give us any new connectivity.)

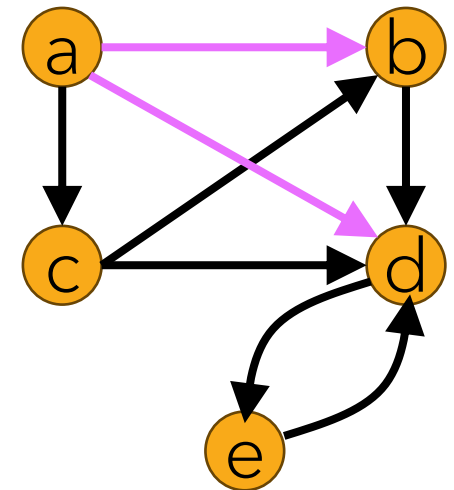
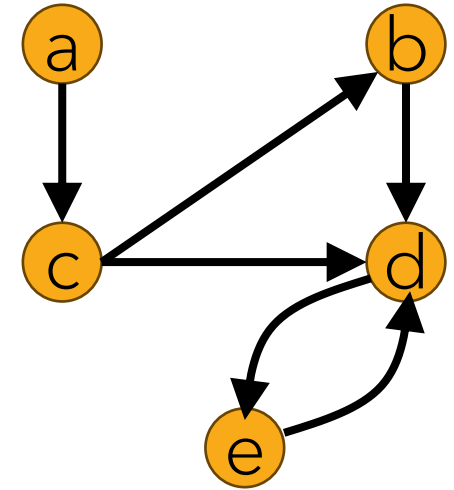


EXECUTING THE DYNAMIC PROGRAM

When $k = 3$ (vertex c) we finally get some action:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Vertex a can reach both b and d using c . This gives:

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$


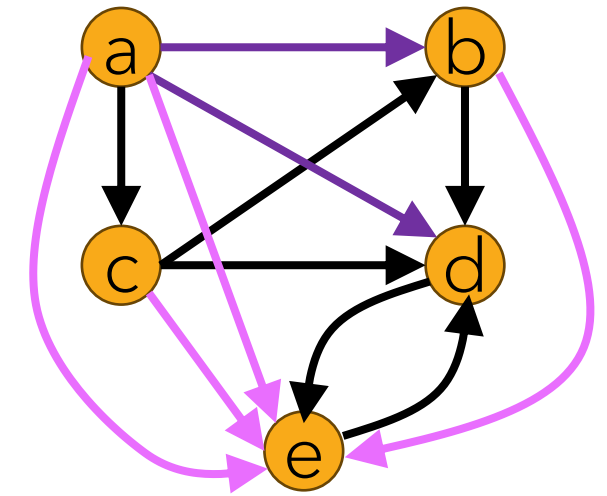
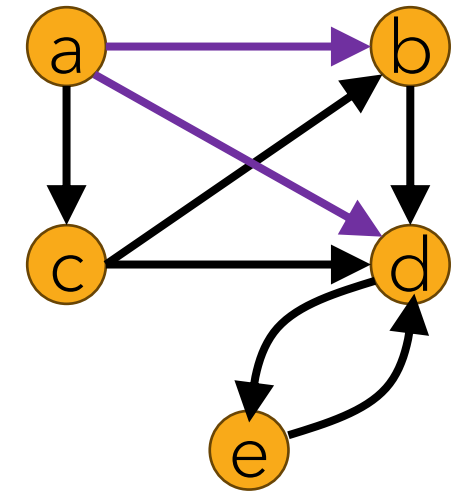
EXECUTING THE DYNAMIC PROGRAM

When $k = 4$ (vertex d) we get more new paths:

0	1	1	1	→ 0
0	0	0	1	→ 0
0	1	0	1	→ 0
0	0	0	0	1
0	0	0	1	→ 0

Every node can reach vertex e via vertex d .

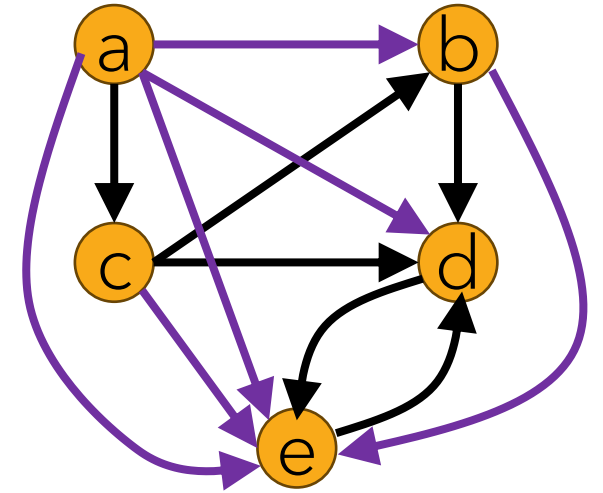
0	1	1	1	1
0	0	0	1	1
0	1	0	1	1
0	0	0	0	1
0	0	0	1	1



EXECUTING THE DYNAMIC PROGRAM

Finally $k = 5$ (vertex e):

0	1	1	1	1
0	0	0	1	1
0	1	0	1	1
0	0	0	0	1
0	0	0	1	1



Nothing exciting - all of these nodes were already reachable. Thus, this is our final matrix, representing the transitive closure of this graph.

TRANSITIVE CLOSURE - PSEUDOCODE

```
def TC(A):  # A is the nxn adjacency matrix
    R[0] = A
    for k = 1 to n:
        for s = 0 to n-1:
            for f = 0 to n-1:
                combine = R[k-1][s][k] & R[k-1][k][f]
                R[k][s][f] = R[k-1][s][f] | combine
    return R[n]
```

Note the bitwise operations above. Also, this is written to make a new $n \times n$ matrix for each $R[k]$, but it doesn't have to be – we can work entirely in place on a single matrix.

THE FLOYD-WARSHALL ALGORITHM

The previous algorithm was published by Stephen Warshall in the *Journal of the ACM* in 1962; it was explicitly referenced by Robert Floyd when he [published](#) his all-pairs shortest-path algorithm later that year.

Of course, neither of them knew that Bernard Roy had published the same algorithm three years earlier in the proceedings of the French academy of sciences. Nevertheless, this algorithm is most commonly known these days as the Floyd-Warshall algorithm.

FLOYD-WARSHALL - PSEUDOCODE

I am giving here a slightly modernized translation of what Floyd published in *Communications of the ACM* in 1962.

```
def FW(W):          # W is the nxn weight matrix; if no
    d = W           # direct edge exists, weight =  $\infty$ 
    for k = 1 to n:
        for s = 0 to n-1:
            for f = 0 to n-1:
                combine = d[s][k] + d[k][f]
                if combine < d[s][f]:
                    d[s][f] = combine
    return d
```

THE FLOYD-WARSHALL ALGORITHM

Note the extreme similarity between Warshall's (Roy's) transitive closure algorithm and Floyd's shortest-paths algorithm: It's the same triply-nested for loop, examining the same matrix entries each step of the way.

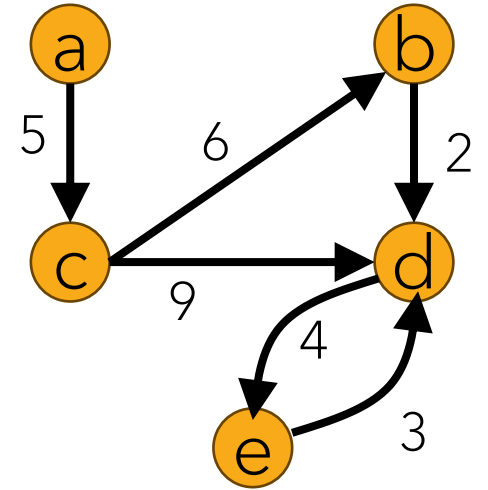
Really, the only difference is that TC is doing bitwise OR and AND, while APSP is adding lengths and then taking a minimum.

The asymptotic runtime of both algorithms is $O(V^3)$, but the extreme simplicity means that the constants are pretty small. Cubic time is pretty bad, but it's still better than running the single-source algorithms V times in a row.

EXECUTING FLOYD-WARSHALL

Before we begin, find the weight of the shortest path from *a* to *e* by inspection.

Give your answer in the Submittity poll now.

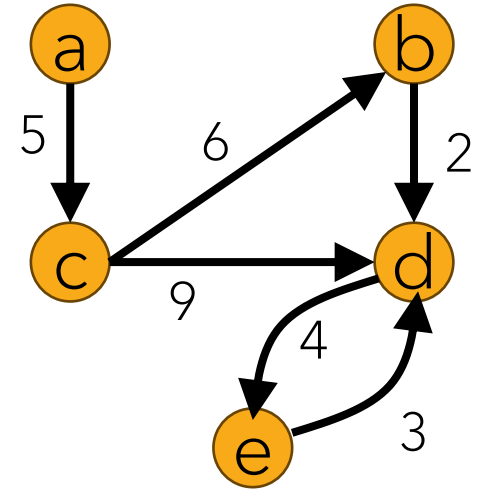


EXECUTING FLOYD-WARSHALL

We start with the adjacency matrix:

∞	∞	5	∞	∞
∞	∞	∞	2	∞
∞	6	∞	9	∞
∞	∞	∞	∞	4
∞	∞	∞	3	∞

As before, our first round is boring – there are no connections that go through *a*.



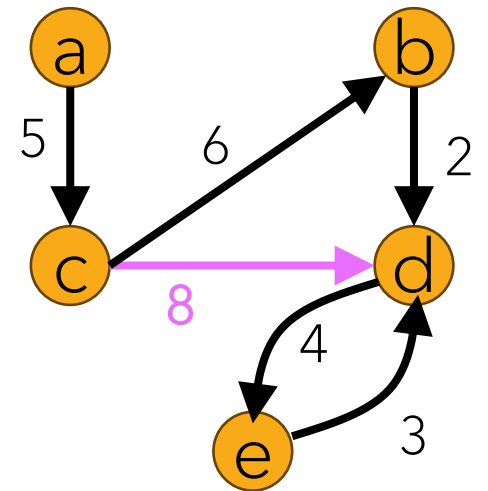
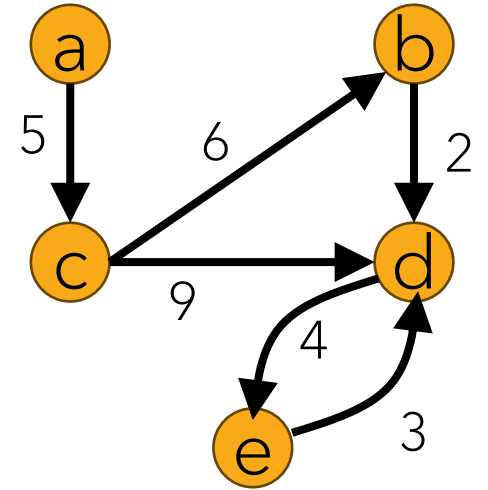
EXECUTING FLOYD-WARSHALL

For the second round, we do need to do some work:

∞	∞	5	∞	∞
∞	∞	∞	2	∞
∞	6	∞	9	∞
∞	∞	∞	∞	4
∞	∞	∞	3	∞

The alternate path is shorter, so we update.

∞	∞	5	∞	∞
∞	∞	∞	2	∞
∞	6	∞	8	∞
∞	∞	∞	∞	4
∞	∞	∞	3	∞



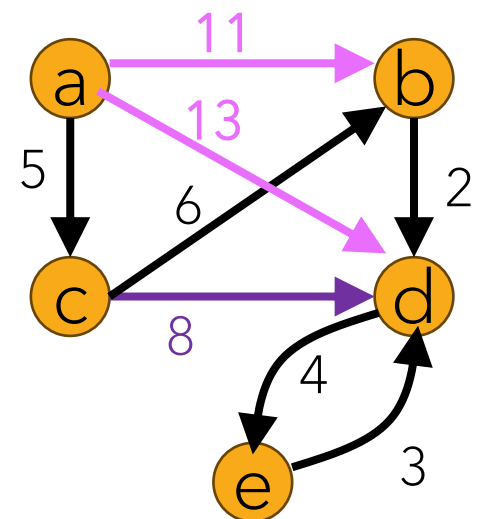
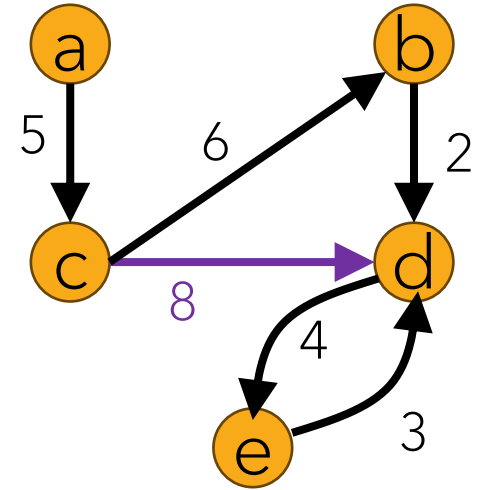
EXECUTING FLOYD-WARSHALL

Third round produces new paths, just as before:

∞	∞	5	∞	∞
∞	∞	∞	2	∞
∞	6	∞	8	∞
∞	∞	∞	∞	4
∞	∞	∞	3	∞

The updated distance matrix:

∞	11	5	13	∞
∞	∞	∞	2	∞
∞	6	∞	8	∞
∞	∞	∞	∞	4
∞	∞	∞	3	∞



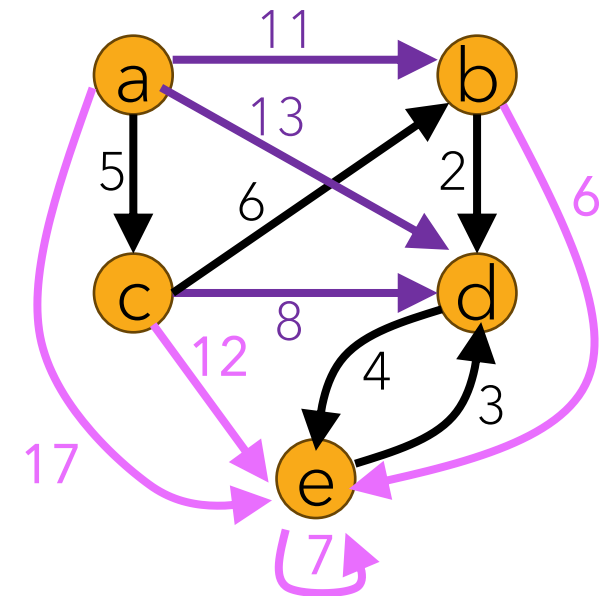
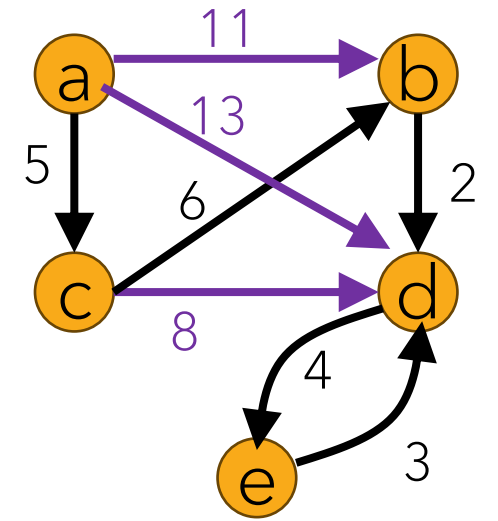
EXECUTING FLOYD-WARSHALL

Fourth round is very busy:

∞	11	5	13	∞
∞	∞	∞	2	∞
∞	6	∞	8	∞
∞	∞	∞	∞	4
∞	∞	∞	3	∞

The updated distance matrix:

∞	11	5	13	17
∞	∞	∞	2	6
∞	6	∞	8	12
∞	∞	∞	∞	4
∞	∞	∞	3	7



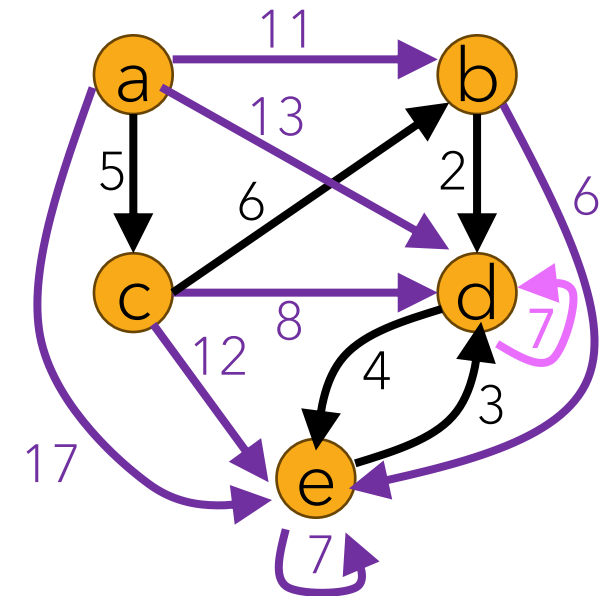
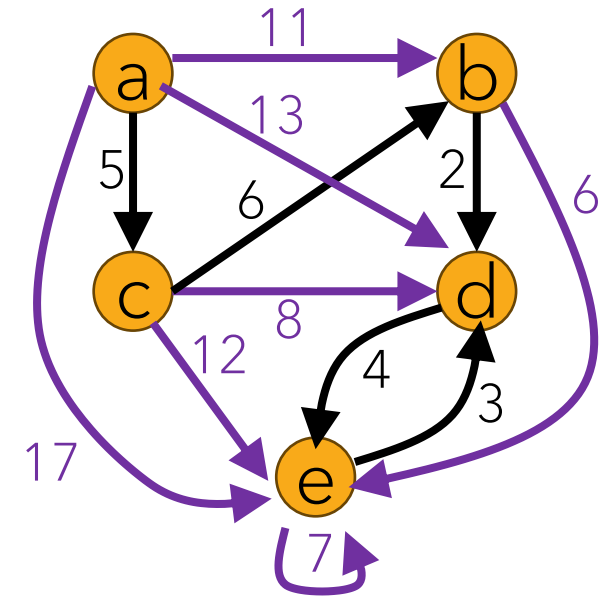
EXECUTING FLOYD-WARSHALL

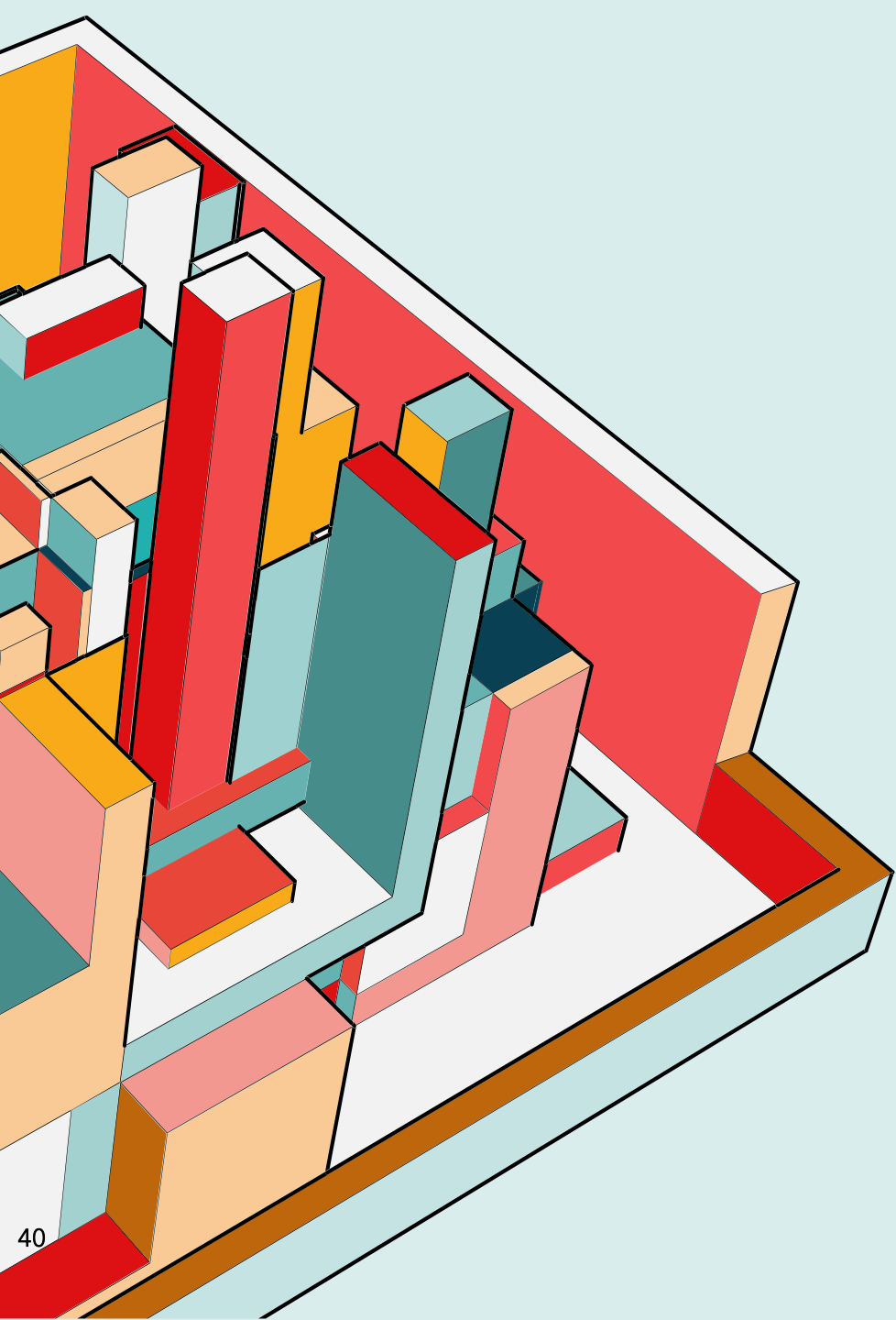
Last round, the only update is from d to d .

∞	11	5	13	17
∞	∞	∞	2	6
∞	6	∞	8	12
∞	∞	∞	∞	4
∞	∞	∞	3	7

The updated distance matrix:

∞	11	5	13	17
∞	∞	∞	2	6
∞	6	∞	8	12
∞	∞	∞	7	4
∞	∞	∞	3	7





EXAM 2 RECAP

BEFORE YOU LEAVE

Please complete this feedback form about today's class.

